Pasted text.txt
TXT
Im having trouble writing my view_history route function for my erg tracker. its supposed to show all previously saved workouts but I dont know where to start. here is previous code to have an idea of what I'm working on. dont write the code for me but point in the right direction

Planning hints for a student's code question.
You've already built most of the pieces you need, so this one is more about assembling them than writing new logic.

1. Look at what you already have. workout_summary(number, workout) turns one workout into one line of text, and every workout lives in state.workouts. So the job of view_history is to call workout_summary once for each workout and put the results on a page.

2. Notice how Page works. Every route returns Page(state, [...]), and that second argument is just a list. You've been writing it out by hand, but nothing stops you from building it step by step: start with a list containing your Header, then add to it, then pass it to Page at the end.

3. Use a loop to fill the list. You wrote a similar loop in find_best_2k. Here, instead of comparing times, each pass through the loop should append a summary string to your page-content list.

4. Handle the numbering. workout_summary needs a number. Think about how to produce 1, 2, 3… while you loop. You could keep a counter variable that you increase each time, or look into Python's enumerate (check what number it starts at by default).

5. Handle the empty case. What should someone see if they haven't logged anything yet? A blank page is confusing. Consider an if that checks whether the list is empty and shows a message like "No workouts yet."

6. Don't forget navigation. Your other pages end with buttons. Add at least a Home button (and maybe "Log a workout") to the list before returning.

Testing tip: Drafter lets you test routes with assert_equal(view_history(some_state), Page(...)). Try writing that test first for a state with two workouts. Writing out the exact Page you expect will make it much clearer what your loop needs to produce.

Looking ahead: your State has a filter_type field and you defined filter_options, so you'll probably want to filter the history later. Getting the basic version working first will make that much easier, since it's just adding an if inside the same loop.

If you get stuck on a specific step, paste what you've tried and I'll help you figure out what's off.

1794523428362_image.png
heres what I have so far, is this in the correct directions

Yes, this is a solid start. You've built the list, looped with a counter, and even added filtering already. A few things to look at:

1. Check where number += 1 sits. It's indented under the for, not the if, so it counts every workout, including ones the filter hides. With the filter set to "Test", you might see "2. Test…" and "5. Test…" with gaps. That might be what you want (each workout keeps the same number no matter the filter), or you might want 1, 2, 3 for whatever is shown. Decide which, and move that line accordingly.

2. You're not returning anything yet. The function still needs to end by giving content to a Page.

3. The empty case. If no workouts match (or none exist), the page will just say "Showing: All" with nothing under it. How could you tell, after the loop, whether anything was added? (Hint: you already have a counter, or you could compare the length of content to what it started at.)

4. Navigation buttons. Add them to content after the loop, before returning.

5. How does the user change the filter? Right now state.filter_type is displayed but nothing on this page lets someone change it. You'll want a SelectBox using your filter_options plus a button that goes to a new route (something like apply_filter) which saves the choice to state.filter_type and then returns view_history(state). That's the same pattern as your save_name route, so use it as a model.

Once you've got the return in, try that assert_equal test with a couple of workouts and different filter values. It'll quickly show whether the numbering behaves the way you want.

elaborate what you mean for 2. and 4

#2: Returning the page

Right now your function builds content and then just stops. In Python, a function that reaches the end without a return gives back None, and Drafter needs a Page to show anything.

Look at your other routes, like log_workout. They all end with return Page(state, [...]), where the second argument is a list written out by hand. The difference here is that you've already built that list and stored it in content. So instead of writing a list inside the Page(...), you pass in the variable that holds it. The state part stays the same as in your other routes.

This should be the very last line of the function, after the loop and after anything else you add to content.

#4: Navigation buttons

Every other page in your app has buttons at the bottom, like Button("Home", index) in log_workout. Without them, the history page is a dead end: the user can see their workouts but has no way to leave.

Since content is a list, you add buttons the same way you added the summaries, with .append(...). The difference is where: do it after the loop finishes (not indented inside it), so the buttons appear once at the bottom instead of after every workout. Think about which pages a user would want to reach from here. "Home" for sure, and maybe "Log a workout."

So the overall order of your function becomes:

Build content with the header (done)
Loop and append summaries (done)
Append buttons
Return the Page with content

Does this look right
1794523557537_image.png

Checking whether the new delete keeps numbering aligned.
Checking whether the new delete keeps numbering aligned.
This looks good, and adding delete was a smart move. It also changes my earlier advice about number += 1: leave it where it is. Because it counts every workout, each number matches the workout's real position in state.workouts (number 1 is index 0, and so on). That means delete will remove the right workout even when a filter is hiding some. If you moved it inside the if, "delete 2" under a filter could delete the wrong one.

A few things to check next:

1. The two new routes. apply_filter and delete_workout need to exist (and be defined with @route) or the buttons won't work. Both follow the save_name pattern: take state plus the parameter whose name matches the box (filter_type, workout_num), update the state, then return view_history(state).

2. Converting the number in delete_workout. Workouts are numbered from 1, but list indexes start at 0. Think about how to turn the user's number into the right index.

3. Bad input in delete. What happens if someone types 0, or 50 when there are only 3 workouts? Python will raise an error. Add an if that only removes the workout when the number is within range.

4. Deleting a 2k PR. If someone deletes the workout that was their best 2k, state.best_2k_seconds still holds the old time. You already wrote find_best_2k for exactly this, so after removing a workout, use it to recompute the best.

5. The empty case. If no workouts are shown, the page jumps straight from "Showing: All" to the filter controls. A short "No workouts to show" message would help. Your counter can't tell you this here (it counts hidden workouts too), so think about comparing len(content) after the loop to its starting length of 2.

6. Starting state. Make sure your start_server sets filter_type to "All". If it starts as "", the filter check won't match anything and the history will look empty.

Pasted text.txt
Pasted text.txt
TXT

write the skeleton for the assert_equal statements for the route functions. I'm just confused on how to format them

Explaining how the test expectations should be structured
Route tests in Drafter all follow the same shape: you call the route with a starting state (plus any form values), and compare the result to the exact Page you expect back.

python
```
assert_equal(
  route_name(State(...starting values...), any_form_arguments),
  Page(State(...values AFTER the route runs...), [
    ...every item on the page, in order...
  ]))
```
Two things trip people up:

The State inside the expected Page is the state after the route runs. If save_name changes the username, the expected State should have the new name.
Create a fresh State(...) in every test. Your routes change the state they're given, so if you store one state in a variable and reuse it, an earlier test can change what a later test sees.
Here are skeletons for your routes. Fill in the blanks (___) and the page items:

python
```
# index: no form arguments, just a state
assert_equal(
  index(State("", [], 0, "All")),
  Page(State("", [], 0, "All"), [
    Header("Erg Workout Log"),
    ___,                  # the greeting text
    "Your name:",
    TextBox("username", ___),
    Button("Set Name", save_name),
    # ...the rest of the buttons, in the same order
  ]))

# save_name: returns index(...), so the expected Page is the HOME page
assert_equal(
  save_name(State("", [], 0, "All"), "Jayden"),
  Page(State(___, [], 0, "All"), [
    # ...what index shows once a name is set
  ]))

# save_workout: form arguments go in the same order as the parameters
assert_equal(
  save_workout(State("", [], 0, "All"), 2000, 8, 0, 30, "Test"),
  Page(State("", [Workout(___, ___, ___, ___)], ___, "All"), [
    Header(___),            # which message should this be?
    "Split: " + ___ + " /500m",
```

```
    "Average watts: " + ___,
    # ...buttons
  ]))

# view_history: start with a couple of workouts so the loop is tested
assert_equal(
  view_history(State("", [Workout(___), Workout(___)], 0, "All")),
  Page(State("", [Workout(___), Workout(___)], 0, "All"), [
    Header("Workout History"),
    "Showing: All",
    ___,                # summary line 1
    ___,                # summary line 2
    "Filter by type:",
    SelectBox("filter_type", filter_options, "All"),
    # ...the rest, in order
  ]))

# apply_filter: returns view_history(...), so the expected Page is the history page
assert_equal(
  apply_filter(State("", [Workout(___), Workout(___)], 0, "All"), "Test"),
  Page(State("", [...], 0, ___), [
    # ...history page showing only Test workouts
  ]))

# delete_workout: check that the list AND best_2k_seconds update
assert_equal(
  delete_workout(State("", [Workout(___), Workout(___)], ___, "All"), 1),
  Page(State("", [___], ___, "All"), [
    # ...history page after the delete
  ]))
```

view_stats, calculator, log_workout and show_split follow the same pattern: show_split takes form arguments like save_workout, and the other three take only a state like index.

When filling these in, you can use your helper functions in the expected page (e.g. workout_summary(1, Workout(2000, 480, 30, "Test")) or format_time(120)), which saves you from typing out long strings by hand. It's also worth writing a second apply_filter test where a filter hides a workout, and a delete_workout test where you delete the 2k PR, since those are the trickiest behaviors.